RSA2048 Key Generation and Attacks

Paul Joo-Hyun Kim (pauljoohyunkim@gmail.com)

November 2, 2026

RSA

Goal

Generate public and private key pairs for RSA cryptosystem.

RSA Cryptosystem

Nutshell

- Public: n, e, CT
  - n is product of two large primes.
  - e is public exponent, often 0x10001=65537
  - CT is cipher text.
- Private: PT, d
  - PT is plain text.
  - d is private exponent, used for decryption.

where

$$CT \equiv PT^e \mod n \quad (1)$$

$$PT \equiv CT^d \equiv (PT)^{ed} \mod n \quad (2)$$

Generation Steps

Steps

1. Choose two 1024-bit primes p and q.
2. Compute $n = pq$.
   - Output n.
3. Compute $\phi(n) = (p-1)(q-1)$.
4. Choose e, a public exponent such that $\gcd(e, \phi(n)) = 1$
   - In HSM: $e = 65537$
5. Compute $d \equiv e^{-1} \mod \phi(n)$.

Prime Generation

Idea: Generate and Test

1. Generate 1024bit number $\tilde{p}$ from PRNG.
2. Let $p = \tilde{p}|1$
   - This forces p to be odd.
3. Test if p is prime.

Primality Test

- Small Prime Division Test
- Fermat Primality Test
- Miller-Rabin Test

Small Prime Division Test (HYPER-OPTIMIZED!!!!!!)

Given 1024-bit big number p (`uint8_t p[128]`), want to check if $p \equiv 0 \mod \{3, 5, 7, \cdots, 251\}$.

Idea: Step 1. Reduction

p is actually

$$p = 256^{127} p[0] + 256^{126} p[1] + \cdots + 256^{1} p[126] + p[127]$$

For division test with 5 and 13, note that $256 \equiv 1 \mod \{5, 13\}$
So,

$$p \equiv \hat{p} \equiv p[0] + \cdots + p[127] \mod \{5, 13\}$$

where $\hat{p}$ is `uint8_t`.

Idea: Step 1. Reduction

Generally, if $256 \equiv f \mod q$,

$$p \equiv \hat{p} \equiv f^{127}p[0] + f^{126}p[1] + \cdots + f^{1}p[126] + p[127] \mod q$$

Idea: Step 1.1. Horner's Rule

To compute the above, compute $\hat{p} \leftarrow f \times \hat{p} + p[i]$ for $i = 0, \cdots, 127$. Effectively,

$$\hat{p} = \cdots (f \times (f \times (p[0]) + p[1]) + p[2]) \cdots + p[127]$$

computing polynomial in $O(n)$.

Idea: Step 1.1. Horner's Rule

To compute the above, compute $\hat{p} \leftarrow f \times \hat{p} + p[i]$ for $i = 0, \cdots, 127$. Effectively,

$$\hat{p} = \cdots (f \times (f \times (p[0]) + p[1]) + p[2]) \cdots + p[127]$$

computing polynomial in $O(n)$.

Carry

$\hat{p} \leftarrow f \times \hat{p} + p[i]$ results in carry. Remember to divide by q in each process not to go over "double" of the size of the variable.

Idea: Step 2. Multiply by Precomputed Modulo Inverse (Theoretical)

To see if `uint8_t` variable $\hat{p}$ is divisible by 5, note that $205 \equiv 5^{-1} \mod 256$. So,

$$\hat{p} \times 205 \equiv \hat{p}5^{-1} \mod 256$$

then, check to see if it is divisible by 5.

$$\hat{p} \times 205(\%256) \leq \frac{256-1}{5} = 51$$

In general, to check if `uint8_t` $\hat{p}$ is divisible by q,

1. Precompute $r \equiv q^{-1} \mod 256$ in `uint8_t`.
2. Check if $p \times r(\%256) \leq \frac{256-1}{q}$.

Fermat Primality Test

Fermat's Little Theorem

If p is prime and a is not divisible by p,

$$a^{p-1} \equiv 1 \mod p$$

Fermat Primality Test

Check if

$$2^{p-1} \equiv 1 \mod p$$

If false, it is definitely not a prime number. (Does **not** mean it is definitely prime.)

Miller-Rabin Test

For k iterations, error rate is $\frac{1}{4^k}$.

```
Input #1: n > 2, an odd integer to be tested for primality
Input #2: k, the number of rounds of testing to perform
Output: "composite" if n is found to be composite, "probably prime" otherwise
```

```
let s > 0 and d odd > 0 such that n − 1 = 2^s d  # by factoring out powers of 2 from n − 1
repeat k times:
    a ← random(2, n − 2)  # n is always a probable prime to base 1 and n − 1
    x ← a^d mod n
    repeat s times:
        y ← x^2 mod n
        if y = 1 and x ≠ 1 and x ≠ n − 1 then # nontrivial square root of 1 modulo n
            return "composite"
        x ← y
    if y ≠ 1 then
        return "composite"
return "probably prime"
```

Prime Generation Probability

Prime Number Theorem

$$\pi(x) \sim \frac{x}{\log x}$$

Generate-Odd-and-Check (Empirical)

Suppose X the number of generation of 1024-bit odd number until prime. Then,

$$\begin{aligned} X &\sim \text{Geom}(p) \\ p &\approx 0.0027 \\ E(X) &\approx 365 \\ \sigma(X) &\approx 362.8 \end{aligned}$$

n Computation

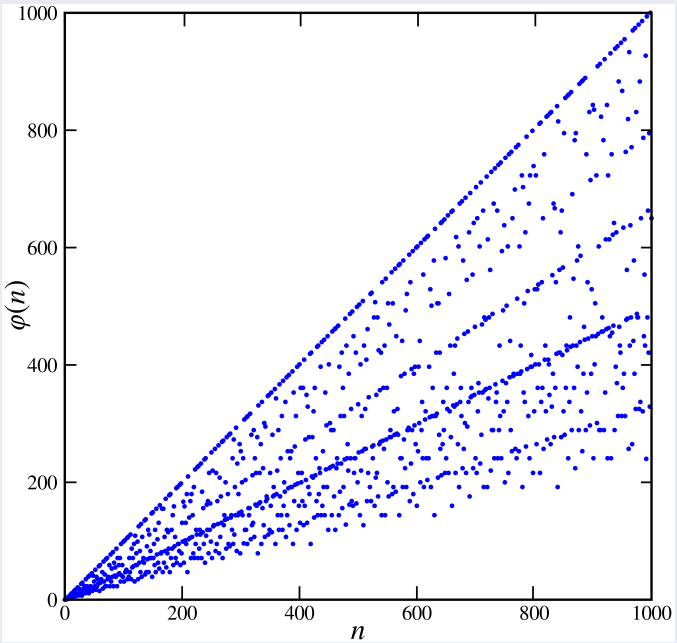
n

Since $n = pq$ and p, q are 1024bit, n is 2048bit.
No carry.

$\phi(n)$ Computation

Euler Totient Function

$\phi(n)$: Number of positive integers relatively prime to n. Note that $\phi(p) = p - 1$ if p is prime.



$\phi(n = pq)$ Computation

Since p and q are prime, it is trivial to compute

$$\phi(pq) = (p-1)(q-1)$$

But it is difficult to compute $\phi(pq)$ without knowing p, q.
Note that $\phi(pq)$ is even (unless $p = 2$ or $q = 2$)

(Slight) Optimization

Since p and q are odd, simply toggle the last bit before multiplication, then recover.

Choice of Public Exponent

Choosing e such that $\gcd(e, \phi(n)) = 1$

Need e such that is coprime to $\phi(n)$.
Since e is public, it can be chosen freely according to condition.
Common choice is $e = 65537 = 2^{2^4} + 1$, a Fermat prime (easy to optimize).

Fermat Prime

If $2^k + 1$ is a prime number, then it must be that $k = 2^m$ for some m, hence a Fermat number.
There are only five Fermat primes that are known: 3, 5, 17, 257, 65537

NIST:FIPS-186-5

1. The public verification exponent e **shall** be selected prior to generating the primes, p and q, and the private signature exponent d.
2. The exponent e **shall** be an odd positive integer such that:

$$2^{16} < e < 2^{256} \tag{3}$$

Computing Private Exponent

Private Exponent

$$d \equiv e^{-1} \mod \underbrace{\phi(n)}_{\text{even}} \tag{4}$$

Problem: HSM Implementation of `HSM_DRV_bignum_mod_inverse` is Montgomery inversion. **Not applicable**.

Chinese Remainder Theorem

Chinese Remainder Theorem (CRT)

For $q_1, \cdots, q_k$ pairwise coprime integers and $a_1, \cdots, a_k$ integers, there exists x such that $x \equiv a_i \mod q_i$ for each k. This x is unique $\mod (q_1 \times \cdots \times q_k)$

Idea

Write $\phi(n) = 2^k m$ where m is odd.
Since 2^k and m are coprime, use CRT. Inversion mod m is given, and mod 2^k is optimizable.

CRT-based Inversion

$d \equiv e^{-1} \mod \phi(n)$

To compute d,

1. Decompose $\phi(n) = \underbrace{m}_{\text{odd}} \times 2^k$.
2. Find $x_1 \equiv e^{-1} \mod m$ by `HMF_DRV_bignum_mod_inverse`
3. $x_2 \equiv e^{-1} \mod 2^k$ by optimized inversion for $\mod 2^k$.
   - $b_0 = 1$
   - For $i = 0$ to $i = k = 1$,
     - $X_i = b_i \mod 2$
     - $b_{i+1} = (b_i - aX_i)/2$
   - Return $x = (x_{k-1} \cdots x_0)$

$d \equiv e^{-1} \mod \phi(n)$ (Cont'd)

4. Compute $m_1 \equiv \left(2^k\right)^{-1} \mod m$ by `HMF_DRV_bignum_mod_inverse`
5. Compute $m_2 \equiv m^{-1} \mod 2^k$
6. Return $x_1 \times 2^k \times m_1 + x_2 \times m \times m_2 \mod \underbrace{\phi(n)}_{m2^k}$

NIST:FIPS-186-5

In the extremely rare event that $d \leq 2^{nlen/2}$, then new values for p, q, and d **shall** be determined. A different value of e may be used, although this is not required.

Smooth Primes (Exponentially Uncommon) vs Safe Primes (Somewhat Common)

If prime p is such that $p \pm 1 = p_1^{m_1} p_2^{m_2} \cdots p_n^{m_n}$ (Either $+$ or -) where $p_i \leq B$, it is called "B-smooth prime", and may be subject to Pollard's $p-1$ attack.

Choose "safe prime"; prime $2p+1$ such that p is also prime. (p is called Sophie Germain prime)

Example (picoCTF 2022, Hard)

n =
0xb03ea698ce2b51fb00e11e6fbaf1e5373dc5b0c70eb2b14a36d21e86
67be8774eee51f6050a10237f6b24f21204fc8013681e7ed72ed051188f327
4aae8f1de0d39389b514c196fa82c98a270bfabefd044da8c687b0e114ebbd
e82536c0709ac5ad81bfe0077e9d9b798ad5abecee52767e68f8060c459365
21fd93893102eb1676f2ff41324a7a6b3dff2e830538e06d25934e9f14bf6b
40ab5674fe648e314bf06f84282f5ef52bc1401de3a42eb66e64bcdadd2674
348e5bdb7016feda44d719af387a948ad81cbaed10213dd930fc7bc7677d8c
4cdab0645d0ff15e6ad6ca37135942c3be08f23e7be0992c8b3370dcdc3104
5e086d823107fb2e443dc9
c =
0xa913a96e215b5aa79c702d27fa375c73d06787639c4131fb32877caf
efaa8faf70e15f6a17ef2a9a6f5310b157cb287b740e77cb5385081d1853a9
104bc16357b259fa2d146bd87398d4ef6f1c078289812952c67792cf9cd745
...

Pollard $p-1$ Algorithm

1. Let $M = \prod_{\text{prime} q \leq B} q^{\lfloor \log_q B \rfloor}$ where B is a "smoothness bound"
   - B is the upper value to the largest possible prime number factor of $p-1$.
2. Choose a coprime to n. (Can choose $a = 2$)
3. Compute $g = gcd(a^M - 1, n)$
   - Exponentiation can be done $\mod n$
4. Cases:
   - If $1 < g < n$, then g is a factor.
   - If $g = 1$, then choose larger B and start again.
   - If $g = n$, then choose smaller B and start again.

```python
import math
from sympy import isprime
M = 2
for i in range(3,2**16,2):
    if isprime(i):
        M *= i
g = math.gcd(pow(2, M, n) - 1, n)
print(g)
```

✓ 1.0s Python

```
166778678181685622576528273627655558408416154535903271612099666211650718813574353644982679332718
```

16-smoothness took 1 second to break on my laptop.
20-smoothness took about 16 seconds.
In general, exponential time needed.

Franklin-Reiter Related-Message Attack

Dangerous Situation: Related-Message

Suppose for RSA system:
$n = pq, \phi(n) = (p-1)(q-1), e = 0x10001, d \equiv e^{-1} \mod \phi(n)$.
Suppose two ciphertexts C_1, C_2 are given for two plaintext M_1, M_2 where the difference between M_1, M_2 is known.

Example

- Erroneous message (M_1): "dof" $=$ 0x646f66
- Correct message (M_2): "dog" $=$ 0x646f67
- Difference (Δ): $M_2 - M_1 = 1$

Franklin-Reiter Related-Message Attack

Note that

- $C_1 \equiv M_1^e \mod n$
- $C_2 \equiv (M_1 + \Delta)^e \mod n$

Note that $f_1(x) = x^e - C_1$ and $f_2(x) = (x + \Delta)^e - C_2$ have common root $x = M_1$ in mod n.
Therefore, find GCD of $f_1(x)$ and $f_2(x)$ in the form of $x - M_1$.

Remark

- Note that there is no size limit on Δ.
- GCD is computed using Euclidean algorithm.
  - Done modulo n (which is composite) with symbolic manipulation.

Lattice Attack

Lattice Attacks

- Coppersmith's attack
  - Available when e is small and padding is not used.
- Håstad's Broadcast Attack
  - Available when same plaintext, but with different moduli n for different people.
- Boneh-Durfee attack
  - Available when d is known to be much smaller than n.

$\vdots$